

An Energy-Efficient Unstructured Sparsity-Aware Deep SNN Accelerator With 3-D Computation Array

Chaoming Fang¹, Member, IEEE, Ziyang Shen¹, Member, IEEE, Zongsheng Wang¹, Member, IEEE, Chuanqing Wang¹, Member, IEEE, Shiqi Zhao, Member, IEEE, Fengshi Tian², Member, IEEE, Jie Yang¹, Member, IEEE, and Mohamad Sawan¹, Life Fellow, IEEE

Abstract—Deep spiking neural networks (DSNNs), such as spiking transformers, have demonstrated comparable performance to artificial neural networks (ANNs). With higher spike input sparsity and the utilization of **accumulation (AC)**-only operations, DSNNs have great potential for achieving high energy efficiency. Many researchers have proposed neuromorphic processors to accelerate spiking neural networks (SNNs) with dedicated architectures. However, **three problems** still exist when processing DSNNs, including **redundant memory access** among timesteps, **inefficiency** in exploiting **unstructured sparsity** in spikes, and the **lack of optimizations** for new operators involved in DSNNs. In this work, an accelerator for deep and sparse SNNs is proposed with **three design features**: a 3-D computation array that allows **parallel computation of multiple timesteps** to maximize **weight data reuse** and **reduce external memory access**; a **parallel non-zero data fetcher** that efficiently searches non-zero spike positions and fetches corresponding weights to **reduce computation latency**; and a **multimode unified computation scheduler** that can be configured to **maximize energy efficiency** for spiking convolution (**SCONV**), spiking Q , K , and V matrix generation, and spiking self-attention (SSA). The accelerator is implemented and fabricated using 40-nm CMOS technology. When compared

with state-of-the-art sparse processors, it achieves the best energy efficiency of 0.078 pJ/SOP and the highest recognition accuracy of 77.6% on ImageNet using the spiking transformer algorithm.

Index Terms—3-D computation array, deep spiking neural networks (DSNNs), sparse architecture, spiking neural network (SNN) processor, spiking transformers.

I. INTRODUCTION

SPIKING neural networks (SNNs) are receiving increasing attention as potential **complements** to conventional artificial neural networks (ANNs). In SNNs, information are represented using single-bit spikes, effectively replacing multiply and accumulate (MAC) operations with **accumulation (AC)**, as shown in Fig. 1(a). In addition, due to the binary representation of activations in **SNNs**, average **input sparsity** can reach up to **90%** without pruning [1], [2], whereas in **ANNs**, only approximately **50% input sparsity** can be achieved with rectified linear unit (ReLU). These properties make SNNs an **ideal** choice in scenarios with **limited resources**. In the past, the performance gap between ANNs and SNNs has limited the broader application of SNNs, with the stereotype that SNNs can only be applied in small applications, such as digit recognition or simple serial processing [3], [4], [5]. However, recent algorithmic breakthroughs have addressed this challenge. As shown in Fig. 1(b), **modern neural network** architectures, such as **Transformers** and ResNet, have been successfully modified and **applied to SNNs**. With proper training or conversion, spiking transformers can achieve similar performance to ANNs on challenging tasks, including **image recognition** on ImageNet [6], [7], [8], **object detection** [9], [10], **semantic segmentation** [11], and even conversing with people using **spiking large language models (LLMs)** [2]. Furthermore, spiking transformers enable more efficient **implementations** than vanilla transformers, not only due to the intrinsic properties of SNNs, such as binary representation and high sparsity but also due to the **removal** of costly **non-linear operations** like **softmax** in self-attention [7], [8], [11]. With efficient network architectures and minimal performance loss on mainstream artificial intelligence (AI) tasks, deep SNNs (DSNNs) like spiking transformers are getting increasing attention.

To fully exploit the low power potential in SNNs, hardware support to handle the sparse spike operations in SNNs is required. Such processors with specialized datapaths and architecture are called **neuromorphic processors**. Many successful

Received 19 June 2024; revised 27 September 2024; accepted 17 November 2024. Date of publication 4 December 2024; date of current version 26 February 2025. This article was approved by Associate Editor Yoonmyung Lee. This work was supported in part by the STI2030-Major Projects under Grant 2022ZD0208805, in part by the “Pioneer” and “Leading Goose” Research and Development Program of Zhejiang under Grant 2024C03002, and in part by the Key Project of Westlake Institute for Optoelectronics under Grant 2023GD004. (Corresponding authors: Jie Yang; Mohamad Sawan.)

Chaoming Fang is with the Center of Excellence in Biomedical Research on Advanced Integrated-on-Chips Neurotechnologies (CenBRAIN Neurotech), School of Engineering, Westlake University, Hangzhou 310030, China, and also with Zhejiang University, Hangzhou 310007, China.

Ziyang Shen, Jie Yang, and Mohamad Sawan are with CenBRAIN Neurotech, School of Engineering, Westlake University, Hangzhou 310030, China (e-mail: yangjie@westlake.edu.cn; sawan@westlake.edu.cn).

Zongsheng Wang is with the College of Information Science and Electronic Engineering, Zhejiang University, Hangzhou 310007, China.

Chuanqing Wang is with CenBRAIN Neurotech, School of Engineering, Westlake University, Hangzhou 310030, China, and also with the Institute of Energy, Hefei Comprehensive National Science Center (Anhui Energy Laboratory), Hefei 230088, China.

Shiqi Zhao is with CenBRAIN Neurotech, School of Engineering, Westlake University, Hangzhou 310030, China, and also with the School of Computer and Communication Engineering, Northeastern University, Shenyang 066004, China.

Fengshi Tian is with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, SAR.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/JSSC.2024.3507095>.

Digital Object Identifier 10.1109/JSSC.2024.3507095

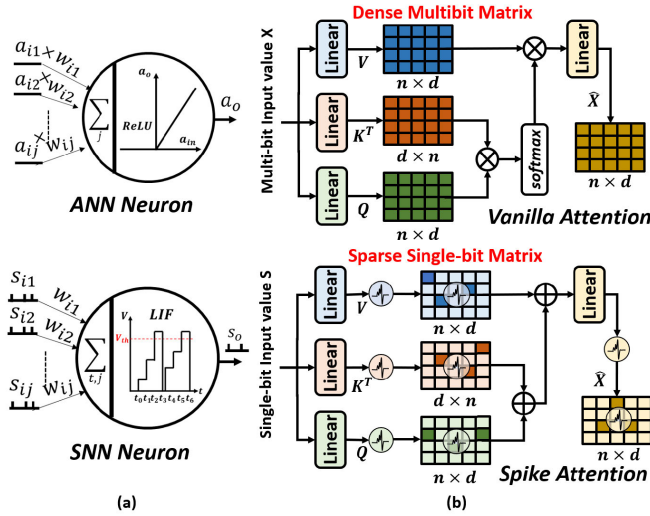


Fig. 1. ANN and SNN neuron models. (a) MAC is required for ANN while only AC is needed for SNN. (b) Vanilla attention and spiking attention.

implementations have been done in recent years [1], [2], [3], [4], [5], [6]. Nevertheless, mapping DSNNs on existing neuromorphic processors faces **three main challenges**, as shown in Fig. 2. The first challenge is the **redundant memory access across timesteps**. The forward path of SNN is recurrent, and thus, the complete inference requires traversing the network for T times, where T is the timestep length. Existing neuromorphic processors adopt a straightforward implementation to process each timestep sequentially. **Weights are identical in each timestep and thus loaded repeatedly from off-chip memory repeatedly** because on-chip weight buffer size is limited. Partial sums will also be moving between off-chip and on-chip memory because of the data dependency of neuron states in each timestep. These redundant memory access are costly and become dominant when the network gets deeper and larger, and the additional latency and energy consumption brought by this issue may even offset the energy efficiency gains obtained from single-bit computation.

The second challenge is the high overhead of exploiting the **unstructured sparsity** in SNN. A typical way to exploit sparsity is to do **sparse matrix multiplication** [12], [13]. The non-zero elements will be first **compressed into sparse format** such as compressed sparse row (CSR) or coordinate format (COO). The elementwise multiplication is done between the **compressed vector and the output coordinates of each computed result can be obtained**. However, **applying such a method to SNN faces two problems**. First, **compressing SNN input into existing compressive formats takes up extra memory space**. It takes multiple bits to represent the coordinates or pointers in existing formats, but it takes only 1 bit to present a spike originally. In addition, the output positions of each spike input are determined during runtime and they are irregular. In existing sparse processors, **large output buffers will be implemented** to allow the output values to be accumulated on chip [12], [14]. This large buffer is costly in SNN processors because the membrane potentials are only intermediate values and will become single-bit spikes after firing.

The final challenge is the **lack of optimization for different operators** involved in DSNNs. Compared with conventional

SNNs, DSNNs introduced some new operators, including **spiking convolution** (SCONV), **spiking QKV matrix generation** (S_QKV_Gen), and **spiking self-attention** (SSA). These operators have distinctive features in terms of fan-in level, data reuse, transpose, and operand number involved. Existing neuromorphic processors are mainly designed for simple SNNs consisting of only MLPs, and it will be inefficient to map new operators on these processors [3], [4], [5]. It is still a challenge to consider how to optimize for the new operators involved in state of the art (SOTA) SNNs with the least resource overhead.

To **overcome these challenges**, this work proposed a **40-nm unstructured sparsity-aware spiking attention and convolution accelerator**. It achieved an **energy efficiency of 0.078 pJ/SOP** and **77.6% accuracy on ImageNet** using spiking transformers. The accelerator has three main design features as follows.

- 1) A **3-D computation array** with timestep-paralleled adder trees to maximize computation throughput and on-chip data reuse across different time steps. It allows parallel computation across different timesteps and therefore reduces the external memory access of both weights and partial sums.
- 2) An **unstructured non-zero fetcher** that efficiently loads valid **spike-weight pairs** to the computation array to reduce computation latency. A **two-stage fan-in** prioritized sparse dataflow is proposed to alleviate the problem of storing multi-bit indexes and implementing large output buffers.
- 3) A **multi-mode unified computation scheduler** to maximize the efficiency of each operator. It includes a **core memory router** to avoid input buffer bank conflict in SCONV, a **bidirectional compressor** to facilitate efficient data transpose operations for S_QKV_Gen, and a **sparse attention skipper** (SAS) to predict and skip zero output for SSA.

II. BACKGROUND

Fig. 3 shows the network architecture of a spiking transformer. In this work, we mainly apply a spiking transformer to image processing, and therefore, the structure of the network is based on **Visual Transformers** (ViTs) [15]. The **spiking tokenizer**, which consists of multiple spiking conv2d layers, **converts the input images into sequences of spiking tokens** to enable the transformer model to process visual information in a tokenized format. Unlike the tokenizer in ViT, **convolution with stride is used to downsample** to image and no positional embedding is required, making the computation in the tokenizer fully spike-based. The key building block for **spiking transformers** is spiking attention, which consists of mainly two parts: S_QKV_Gen and dot-product SSA computation. S_QKV_Gen is a linear layer that converts input sequence $X \in \mathbb{R}^{T \times N \times D_x}$ into spike form $Q, K, V \in \mathbb{R}^{T \times N \times D}$. Also, the subsequent SSA computation is shown in the following equation:

$$\text{SSA}(Q, K, V) = \text{ConvBN}(\text{SN}(QK^T V \times s)) \quad (1)$$

where s is the scaling factor. The main difference between spiking and vanilla self-attention is that it eliminates softmax computation, leaving **spike-only matrix multiplications**. Because of this, it allows the change of computation order to

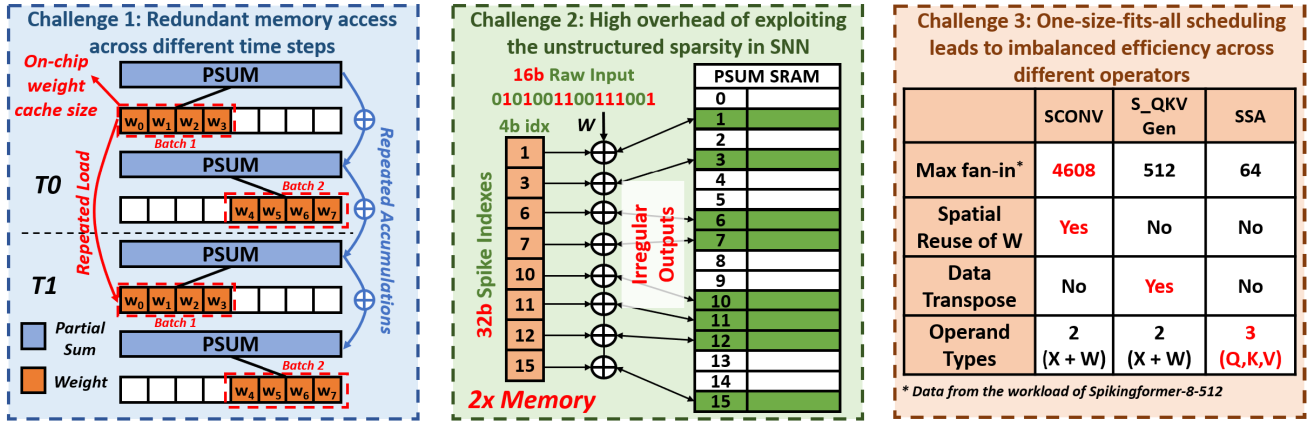


Fig. 2. Three challenges of mapping DSNNs on neuromorphic processors: 1) redundant memory access in different timesteps; 2) large overhead of exploiting unstructured sparsity; and 3) limited support for various operators in DSNNs.

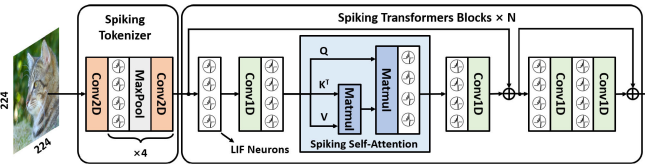


Fig. 3. Overview of the spiking transformers algorithms. It consists of consecutive layers of convolution spiking tokenizers and self-attention blocks.

compute $K \cdot V$ rather than $Q \cdot K$ first to reduce computation complexity when $N > d$ [8], where N is sequence length and d is the embedded dimension of the network. The computation complexity will decrease from $O(N^2d)$ to $O(Nd^2)$. In addition, all operators, including SCONV, linear, and attention, take a 1-bit spike as input by placing leaky integrate fire (LIF) neurons in front of each operator. This ensures that no integer multiplication will be involved in the network to ensure the network can be well supported in neuromorphic processors [11], [16]. Only the first convolution layer is a non-spike layer, and its function is to convert multi-bit images into spike sequences.

III. OVERALL ARCHITECTURE

Fig. 4 shows the block diagram of the proposed accelerator. It comprises three computing clusters, each cluster contains three sparse computing cores, and these cores share a computation scheduler. Weight buffers are implemented to be local to each sparse computing core, while input spike and output spike memory are implemented to be shared by three cores in a cluster.

The accelerator has three main design features. The first one is the 3-D computation array. The array takes weights and spikes and generates AC results of the weights. It adds an additional dimension to the conventional 2-D accelerator array, which reuses weight and allows parallel computation among different timesteps. The array is implemented with adder trees, and there is a total of 8×8 adder trees with eight input ports. A neuron state update block is used to handle the state dependencies.

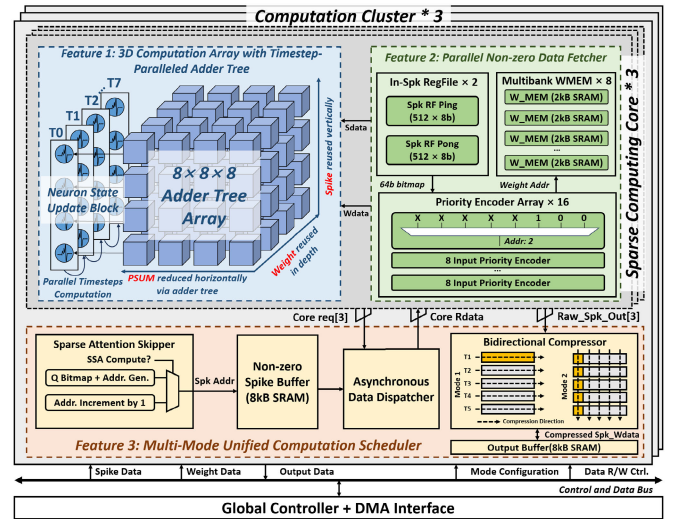


Fig. 4. Overall architecture of the chip with three key features: 1) 3-D computation array to allow parallel computation of different timesteps; 2) a non-zero data fetcher that continuously fetches non-zero spike-weight pairs; and 3) a unified scheduler to support various operators.

The second design feature is the non-zero data fetcher. It will analyze the input spikes, fetch out weight data that correspond to non-zero spike positions from the weight buffer, and send them to the 3-D computation array. To avoid bank conflict, a multi-bank SRAM is implemented to cache weights candidates. Each bank can be accessed independently, and the addresses to each bank are generated using priority encoders. The encoders will take spike bitmaps as input and encode them into addresses consecutively.

The final design feature is the unified computation scheduler, which is responsible for maximizing the computation efficiency of multiple operators involved. By default, compressed spike bitmaps are read from the spike buffer and broadcast to all three cores in the cluster. Raw output spikes are compressed and stored in the output buffer. We insert three modules to this default dataflow, including an SAS for saving operations in SSA, an asynchronous data dispatcher to allow each core to access data without waiting in SCONV, and a

bidirectional compressor to realize efficient data transpose in **S_QKV_Gen**.

IV. TIMESTEP PARALLELED 3-D COMPUTATION ARRAY

A. Timestep-Paralleled Dataflow

The processing procedures of a neuron j in layer l of timestep t in SNN can be written as follows:

$$V_j^l(t) = \sum_{i=0}^N s_i^{l-1}(t) \cdot W_{ij} + \alpha V_j^l(t-1) \cdot (1 - s_j^l(t-1)) \quad (2)$$

$$s_j^l(t) = \begin{cases} 1, & V_j^l(t) \geq \theta \\ 0, & V_j^l(t) < \theta \end{cases} \quad (3)$$

where s denotes input or output spikes, W denotes the weights, V denotes the membrane potentials, and α denotes the leaky factor. Conventionally, the **processor traverses from the first to last timestep sequentially**, because the computation of each timestep is **dependent on V and s of the previous timestep**. However, this will incur **excessive data movement** as mentioned above.

We propose a **timestep paralleled dataflow** as shown in Fig. 5 to alleviate this problem. We notice that the process of each timestep can be divided into two parts, which are **weight accumulation (WA)** and **membrane potential update (MU)**. The WA part refers to $\sum_{i=0}^N s_i^{l-1}(t) \cdot W_{ij}$ in (2), and this part has no data dependency with the previous timestep. The membrane potential update part adds the results of WA to the membrane potential of the previous timestep $V_j^l(t-1)$, and this part is where data dependency happens. In timestep paralleled dataflow, the WA stage of multiple timesteps is executed in parallel. After that, the **AC result of timestep t** will be the input to MU part of timestep $t+1$, and traverse to the final timestep sequentially to obtain output spikes of each timestep. The proposed dataflow allows **parallel computing of different timestep**, which not only **increases data reuse of weights** but also **avoids loading or storing membrane potential** by reducing them in the timestep direction completely **on chip**. As a result, a 68.3% of the external memory access can be reduced compared with timestep sequential dataflow.

B. 3-D Computation Array

Fig. 6 shows the structure of the 3-D computation array. The “3-D” concept denotes the three dimensions that data reuse can be exploited in this computation array. We define the size of the array as (A_D, A_H, A_W) , denoting depth, height, and width, respectively. **Weight reuse occurs in the A_D direction**, allowing parallel computation of **WA with input spikes from multiple timesteps**. **Spikes are reused vertically in A_H dimension** and shared among weights from different output channels in the fan-out direction. **Horizontal reduction of partial sums** is achieved using the **adder tree** to accumulate weights from different input channels in the fan-in direction, in the A_W dimension. Most existing accelerators that adopted the 2-D spatial array exploit the latter two data reuse dimensions [1], [2], [6]. The third timestep dimension added in this

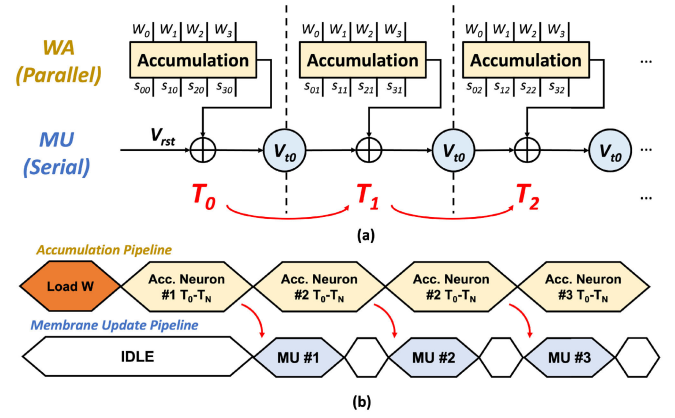


Fig. 5. (a) Illustration of timestep paralleled dataflow with parallel WA and serial MU. (b) Pipeline of WA and MU and **hiding latency** of MU in WA.

accelerator enables parallel timestep acceleration as discussed in Section IV-A. The maximum operating frequency of the 3-D array is inversely proportional to the size of the array due to the longer path in data reduction and more complex routing. We selected the configuration of $A_D = A_H = A_W = 8$ because it achieves the best balance between speed and area compared with other configurations.

We can dive deeper into the 3-D array to see how each element is connected with each other as shown in Fig. 6. Each row in the array represents an A_H -input adder tree, and therefore, the array is fundamentally a spatial combination of $A_D \times A_H$ masked adder trees. Masked adder tree denotes that each input port of the adder tree is masked by input spikes, selecting either a zero value or a weight value to be accumulated. In each cycle, $A_D \times A_W$ spikes and $A_H \times A_W$ weights will be loaded to the computation array, and generating $A_D \times A_H$ partial sums of membrane potentials in registers files called **V_reg**. Physically, the 3-D array is decomposed into A_D groups of $A_H \times A_W$ adder trees to fit in a 2-D floorplan. Spike and weight RFs are implemented locally to each $A_H \times A_W$ sub-array to **alleviate the route timing** pressure in a flattened floorplan.

Once all the WA is finished, the **serial MU** will start, taking the $A_D \times A_H$ AC results and generating spikes of each timestep sequentially. Although MU is serial, we can still **hide the latency of the sequential update in the parallel AC stage** of the next output position, as shown in Fig. 5(b). It takes only **eight cycles to finish an MU** while in most cases the **latency of a WA takes more than 50 cycles**, which means that the latency of MU can be completely hidden. To ensure that there is **no resource conflict**, we implement two sets of **V_regs**, and they work in a double-buffered manner. **One set of V_regs takes the output from WU**, while another set is used for **caching membrane potentials in MU**, and the **role of the two sets exchanges** with each other in the computation of the next position. The 3-D architecture explores a new dimension to exploit weight reuse for recurrent operations. Compared with the conventional 2-D array without parallel timestep computations, the memory access of weight reduces **by T times**, without compromising throughput by the pipelined execution of parallel WA and serial MU.

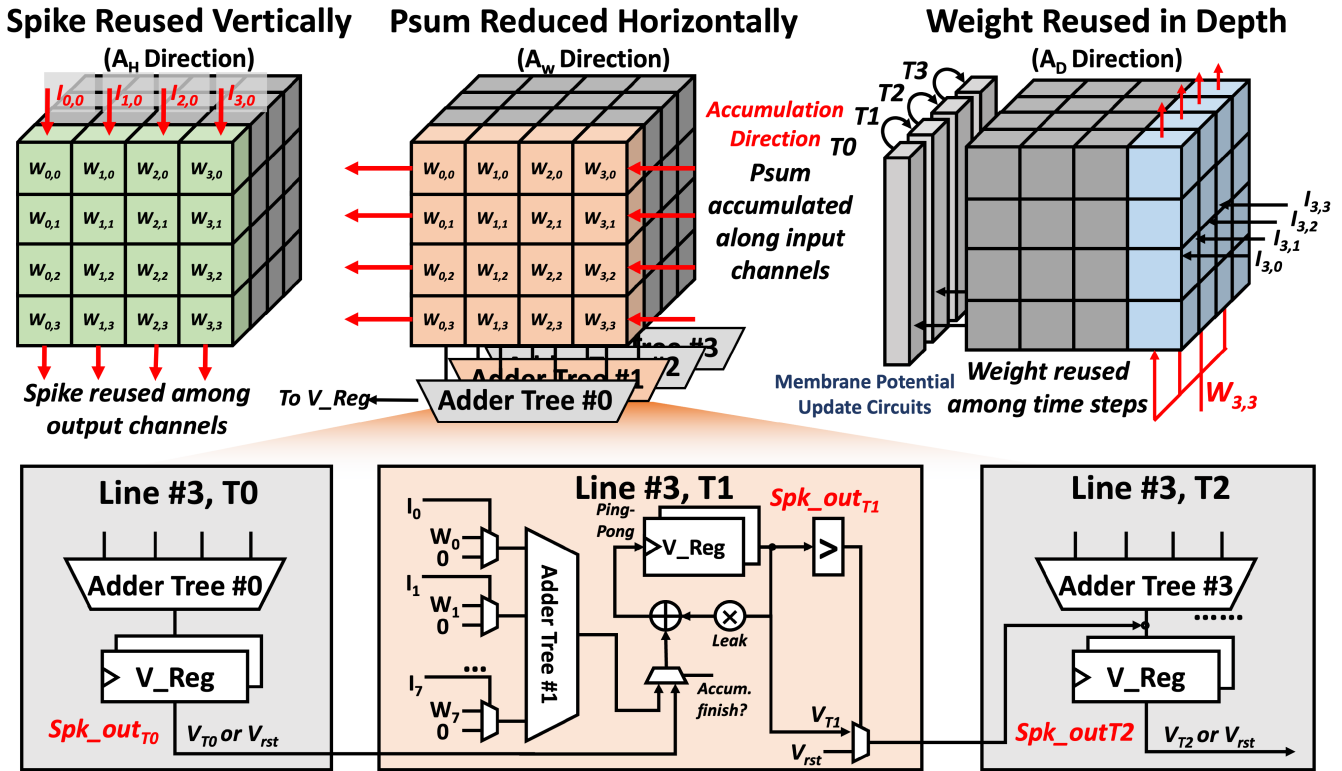


Fig. 6. Three types of data reuse and micro-architecture of a 3-D computation array.

V. UNSTRUCTURED NON-ZERO FETCHER

A. Two-Stage Fan-In Prioritized Sparse Dataflow

As mentioned in the introduction, the existing dataflows adopted in sparse processors for ANN are not applicable for exploiting sparsity in SNN due to two main reasons, which are the storage overhead of existing compressive formats and the overhead of implementing large partial sum buffers on chips. To tackle this, we proposed a dedicated sparse dataflow oriented for SNN as shown in Fig. 7. We use a two-stage bitmap format to represent the sparsity in SNN spikes. In SNN, the activation value of each position on the feature map is spikes with T timesteps. We assign 1 bit for each position, called global bitmap, to represent if all T spikes of each position are 0. If so, a zero value will be set on the global bitmap to represent this. Otherwise, a “1” value is set instead, and the spike information of this position will be stored and called a local bitmap.

With the two-stage bitmaps, the following step is to conduct a fan-in prioritized computation. The proposed dataflow ensures that at each cycle, all weights are accumulated to only one output position. The computation for the next position will not start until all fan-in weights have been accumulated, and output spikes are generated for the previous position. During the AC process of each output neuron, only the weights corresponding to an input neuron with a non-zero global bitmap will be loaded and accumulated to the membrane potentials. After the AC is done, the membrane potential values of the previous position can be discarded, saving the cost of implementing large on-chip buffers.

B. Non-Zero Data Fetcher

Fig. 8(a) shows the structure of the non-zero data fetcher. The non-zero weight address generator generates addresses according to the global bitmap to fetch weights stored in multi-bank weight memories. Local bitmaps and the fetched weight vectors are sent to the 3-D array. The key to reducing latency is to only fetch weight vectors with non-zero global bitmaps. It is thus required to convert the global bitmaps into the addresses of the weight memory. To maximize throughput, it is desirable to generate non-zero addresses every cycle without pauses. We realize this with a dedicated priority encoder with positional reset. The priority encoder will generate the index of the rightmost non-zero bit in the input, which is the global bitmap we defined. The rightmost non-zero bit will be reset to 0 at each cycle. Such a circuit block allows generating multi-bit non-zero addresses from single-bit bitmap during runtime consecutively, which saves memory space and, meanwhile, maintains a high computation throughput. We implemented eight generators with each generator taking 8-bit input and generating eight addresses every cycle. These addresses will be sent to the eight-weight memory banks and fetch out eight weights every cycle without causing bank conflict.

The above methods face a problem of load imbalance. Because the number of weights to be fetched out from each bank is different, and some input ports of the adder tree with fewer non-zero global bitmaps will be idle. We implemented a load balancer between the address generator and weight memory to alleviate this problem. The load balancer will search the neighboring N banks to see if there is an extra available weight queuing for AC when one bank is idle, where

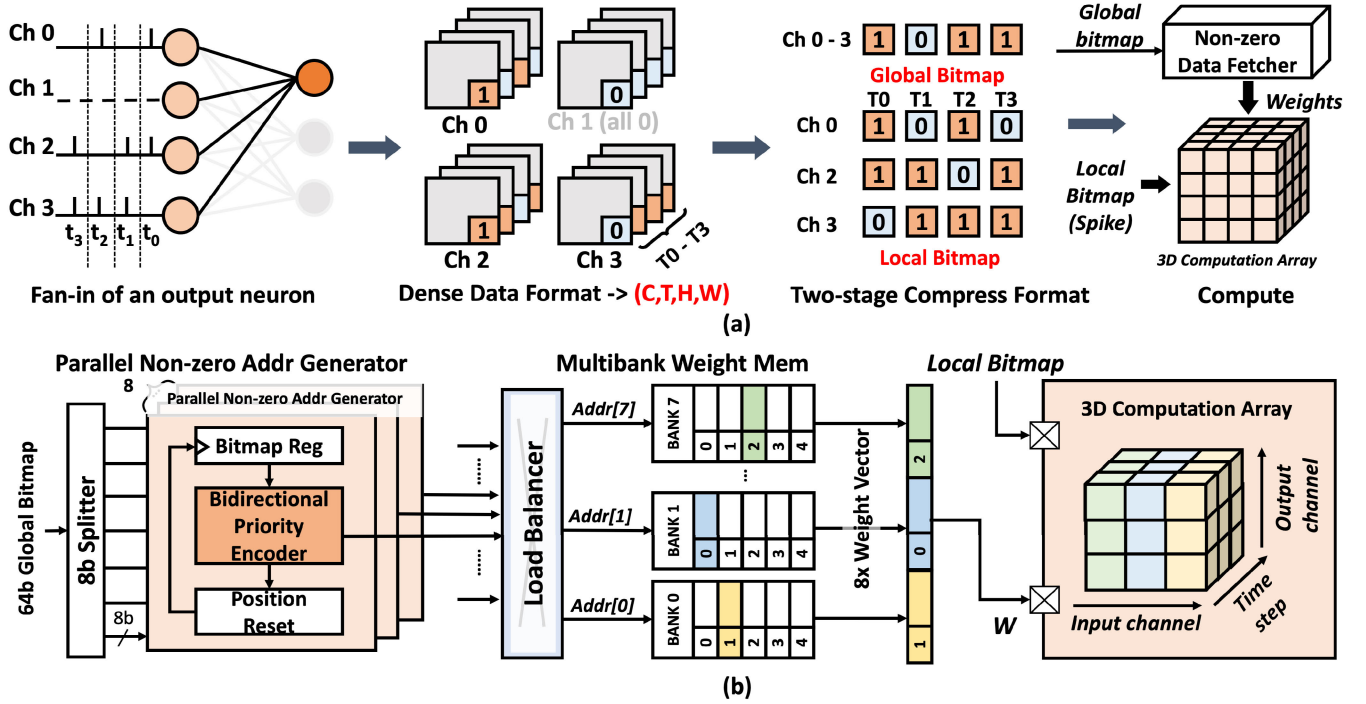


Fig. 7. Two-stage fan-in-prioritized sparse dataflow. (a) Illustration of the global and local bitmaps. (b) Structure of the non-zero data fetcher with address generators, load balancer, and multi-bank weight memories.

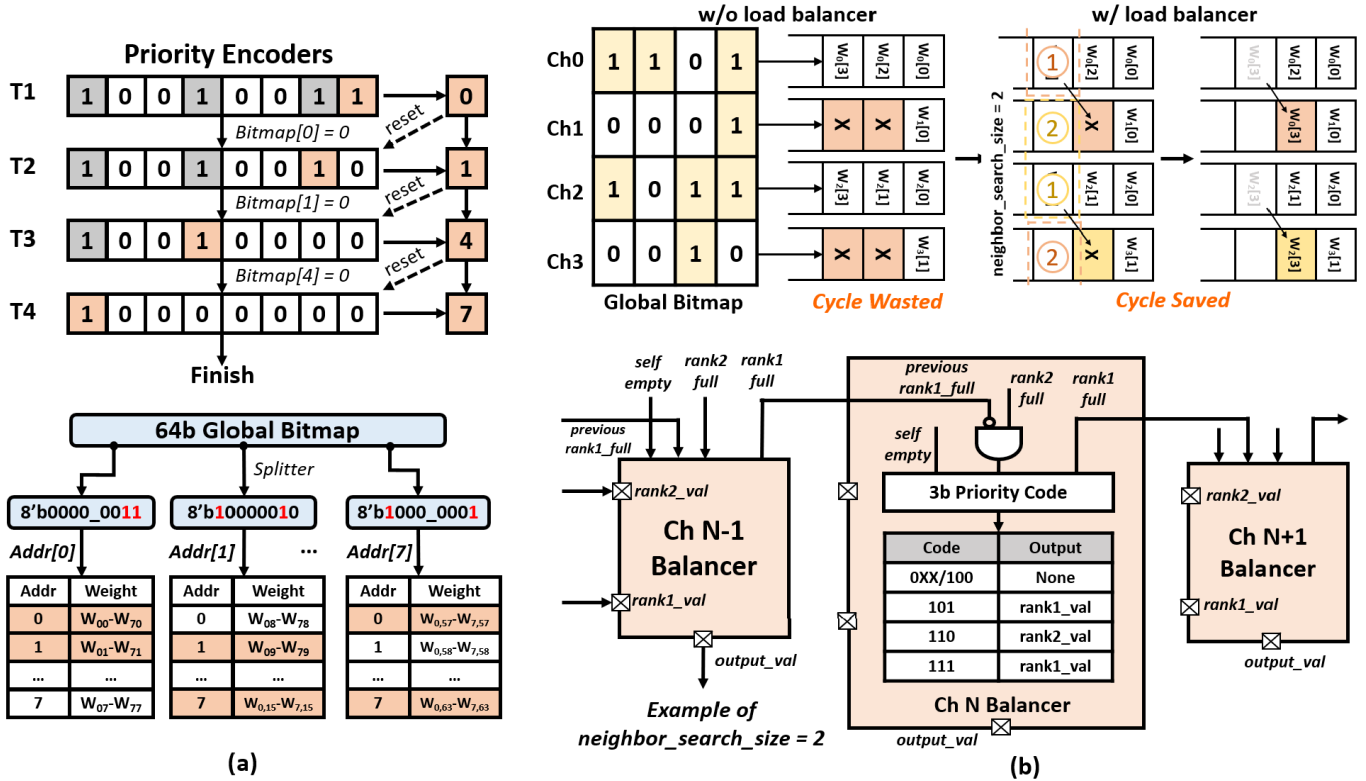


Fig. 8. Components in the non-zero data fetcher. (a) Non-zero address generator to generate conflict-free weight addresses. (b) Load balancer and circuit implementation when search window size = 2.

N is a design parameter of search window size. If so, it will "borrow" this value to the current bank, and feed it to fill the idle adder tree ports. Fig. 8(b) shows an example when $N = 2$. Bank 1 and bank 3 borrow a value from their neighbor

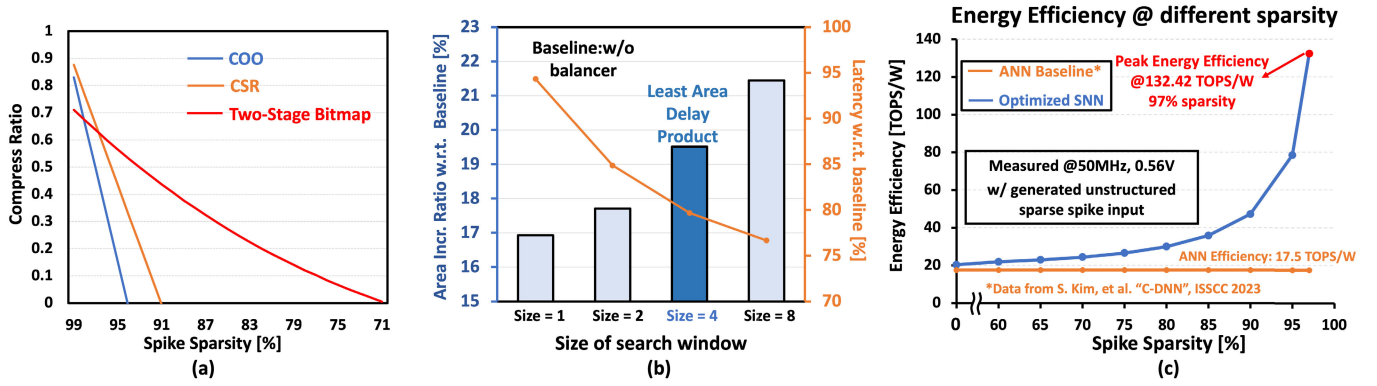


Fig. 9. Measurement results of the non-zero data fetcher. (a) Storage efficiency comparison among COO, CSR, and the proposed two-stage bitmap. (b) Area increment ratio and the latency with respect to the baseline without load balancer of different window sizes. (c) Energy efficiency at different spike sparsities with a higher energy efficiency than ANN baseline at all sparsity.

bank 0 and bank 2, respectively, to save one wasted cycle. The circuitry implementation of the balancer with $N = 2$ is also shown in Fig. 8(b). Each bank has its balancer, and they are cascaded together. To avoid multiple banks requesting the same value, priority codes are assigned in the search window of each bank. In this way, when two banks request the same value, the balancer that assigns a higher priority to that value will win and get the value. As shown in Fig. 8(b), the inversion of rank1_full signal in balancer $N - 1$ will conduct AND with the rank2_full signal in balancer N , indicating that bank $N - 1$ has a higher priority in this position so if this position has available value, it will be allocated to bank $N - 1$ first. The balancer will then arbitrate which neighboring value to select based on the 3b priority codes, as shown in Fig. 8(b). As N gets larger, there are higher chances that an idle bank can get available values to save more cycles, but it will also require more ports and more complex logic to implement the conflict-free balancer. As shown in Fig. 9(b), we estimate the area incremental ratio of varying size N and the latency reduction ratio with respect to a baseline without load balancer. We observe that the best balance between area and latency is achieved when $N = 4$, and thus, this size is implemented in the chip.

The proposed two-stage bitmap format has better storage efficiency than conventional compress formats such as CSR or COO at all sparsity range, as shown in Fig. 9(a). Especially when spike sparsity is low, the compress ratio of CSR and COO decreases rapidly and may even consuming more space than a dense spike format due to the costly storage of multi-bit indexes or coordinates. The energy efficiency gained by the non-zero fetcher is dependent on the spike sparsity, and the evaluation results are shown in Fig. 9(c). We designed a matrix multiplication baseline of two 256×256 matrices with eight timesteps and mapped it on hardware, and we manually tuned the sparsity level by randomly setting zeros in the matrix. The energy efficiency of dense matrix multiplication is 20.2 TOPS/W, and we can observe that when spike sparsity is low (under 60%), the energy efficiency improvement compared with a dense dataflow is limited. This is because sparsity can only be exploited when all timesteps of a neuron are 0, and

this case only gets typical when spike sparsity gets higher. For real SNN workloads, the average spike sparsity of each layer ranges from 69% to 97% according to our experiments and thus can benefit a lot from our proposed zero-skipping approaches.

VI. UNIFIED COMPUTATION SCHEDULER

Fig. 10 shows the structure of the unified computation scheduler. Each cluster has one scheduler shared by three cores. Fundamentally, the scheduler controls that three computation cores take the same vector of spikes in the spike memory, do vector-matrix multiplication (VMM) in the array, and generate output spike data to store back in output memory. But simply decomposing all operators into VMM and applying the same dataflow to them could be inefficient due to the varying computation characteristics of each operator. Therefore, we added some additional modules to facilitate a more efficient operator-specific dataflow on the chip.

The support for SCONV in a unified scheduler is shown in Fig. 10(a). Fan-in prioritize is the key to reducing the store cost of membrane potentials. But in SCONV, each output neuron has a high fan-in level because spikes in a reception field of $3 \times 3 \times \text{Ch_num}$ are all accumulated to the same position. The required weight buffer size in each core will be too large if all fan-in computations are concentrated in one core. Instead, we make each core to be responsible for accumulating input channels in only one position in the receptive field and that requires each core to take different input spikes. Because 3×3 convolution is broadly used in most networks, we implement nine cores in this accelerator to account for the receptive field size of 3×3 conv. Convolutions with shapes larger than 3×3 can also be supported by decomposing larger kernels into multiple smaller kernels when mapping on the chip. For example, 7×7 convolution will break down into four 3×3 kernels, a 1×7 kernel, and a 7×1 kernel. Because in convolution, there is an overlap of the reception field between neighboring output positions. Therefore, a memory router between cores is implemented, allowing each core to fetch spike data not only from the spike memory but also

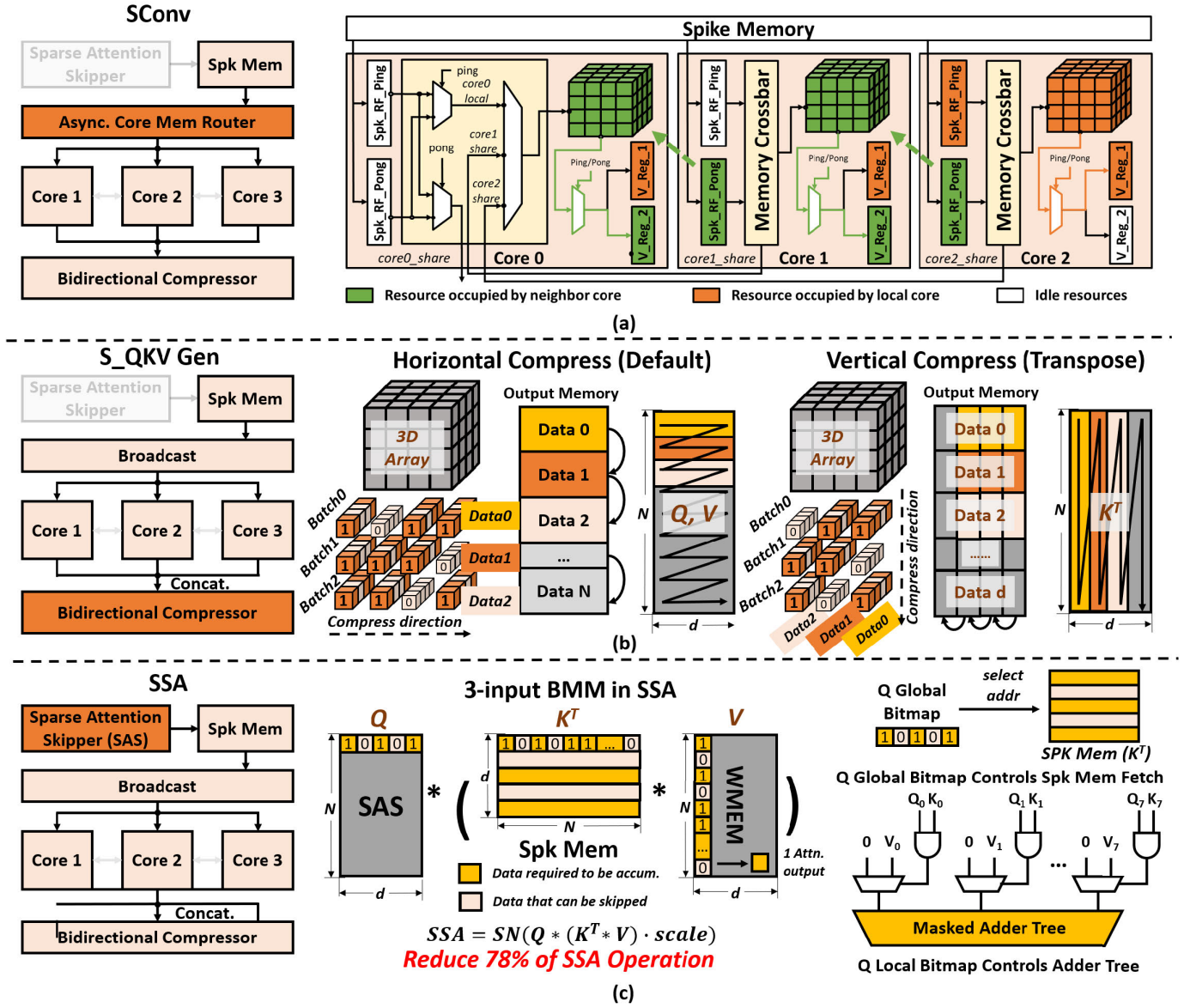


Fig. 10. Three modes of the unified scheduler. (a) SCONV mode uses core memory router to reduce spike memory pressure. (b) S_QKV_Gen mode uses bidirectional compressor to enable fast transpose in K matrix. (c) SSA mode uses SAS to reduce total operations.

from the spike registers in its neighboring cores because the required spikes for a new position are in it. The router can also be configured to fetch data from non-neighbor cores or from spike memory to support convolutions with stride lengths larger than 1. This router alleviates the bandwidth pressure of the spike memory, making the spike memory accessed by only one core at a time and reducing process latency by 51.1%. In addition, we allow each core to asynchronously start the computation of a new position without waiting for other cores to finish. Because in SCONV, input spikes that are allocated to each core are different. The sparsity of these input can be different and thus may lead to different execution times. The old AC results are stored in one set of V_regs in the core waiting for a global AC when all cores finish, while the new partial sums can be stored in another set of V_regs without influencing the old results. The asynchronous

scheduling alleviates the problem of load imbalance among different cores and the memory router eliminates the situation of bank conflict of spike memory, further reducing 8.1% of the processing latency in SCONV.

Another module introduced in the scheduler is called a bidirectional compressor, which is optimized for the data transposition required in S_QKV_Gen. The K matrix needs to be transposed in the subsequent SSA computation, and implementing an additional data transposer for it is not desirable. Instead, the bidirectional compressor controls the data compression direction and the writing logic to the output memory to achieve data transposition with nearly no cost. As shown in Fig. 10(b), there are two modes for the compressor. The default mode is called horizontal mode. Under this mode, the computation array generates A_H output spike vectors with A_D timesteps at each batch. The compressor will identify

which vectors have all zero timesteps, encode it into two-stage bitmaps, and store it continuously in the output memory. In another mode named vertical mode, the compressor still checks and discards all zero timestep spike vectors in each output batch, but it will scatter the remaining spike vectors within the same output batch to pre-assigned non-contiguous locations in the output memory. The spikes in each entry of the output memory will be from different output batches from the computation array. In this way, each entry of the output memory will store one column of the output matrix in the vertical mode, and store one row of it in the horizontal mode, which efficiently controls the transpose option of the matrix.

Finally, for the SSA computation, an SAS is introduced to accelerate the computation. One biggest differences between computing SSA and other operators is that the output of each position is determined by three different data types, including matrix Q , K , and V . As mentioned in Section II, computing $K \times V$ first can reduce total operation numbers when $N > d$. We notice that there is an additional sparsity opportunity that can be exploited when computing SSA. As shown in Fig. 10(c), if the global bitmap of an element in Q matrix Q_{ij} is 0, the entire vector dot product of row j in K^T and matrix V can be skipped. We exploit this additional sparsity opportunity in SAS as follows. First, the global bitmap of one row of the Q matrix is read to the buffer in SAS. The SAS module will generate the non-zero index of the Q global bitmap, which is also the row address to fetch the compressed K^T matrix stored in the spike memory. The V matrix is stored locally in the weight memory of each core, so the sparsity in the K vector can also be exploited similar to other operators. The mask input of the adder trees is bitwise AND of the local bitmaps of Q and K , masking V AC when either the local bitmap of Q or K equals 0. After the selected K row finished dot product with the V , the results stay stationary in V_regs of each core because they can still be accumulated to the final result by fetching out another row of K^T matrix controlled by global bitmaps of Q . This is different from other operators because three matrix are included in SSA computation, and the dot product of one row of K^T and one column of V only contribute partial sums of the final membrane potentials of one output neuron. The next non-zero global bit of Q will fetch the new K^T row and continue accumulating with the same V columns until all non-zero bits in Q have been iterated. The above technique provides a fused computation paradigm for SSA, and it reduces total SSA operations by 78% because of the skip of unnecessary dot product computation between K^T and V .

VII. IMPLEMENTATION RESULTS AND COMPARISON

A. Experimental Setup

The proposed accelerator was fabricated in 40-nm CMOS technology, and the die photograph and the specification of this chip are shown in Fig. 11. We selected the specification of implementing the 3-D computation array with the shape $A_D = A_H = A_W = 8$ and a default weight precision of 8 b. $A_D = 8$ supports eight parallel timestep computations,

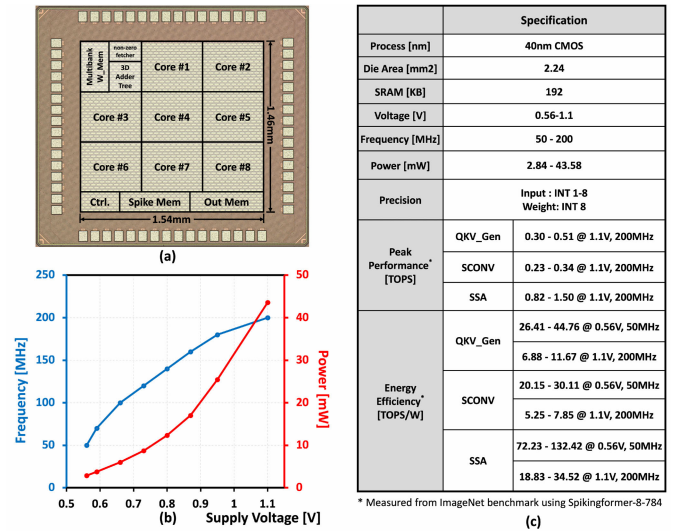


Fig. 11. (a) Die photograph. (b) Voltage–frequency scaling of the chip. (c) Specifications of the proposed accelerator.

which satisfies the need for most directly trained SNN models. A_H and A_D are selected to be 8 to be consistent with the bank number and the bandwidth of the weight memory. Nine cores with one 3-D array in each core are integrated into the accelerators. The chip consumes 1.54×1.46 mm area and runs under the frequency of 50–200 MHz. The chip consumes 2.84–43.58 mW under a 0.56–1.1-V supply voltage. The voltage–frequency scaling is also shown in Fig. 11. The best energy efficiency is 20.28 TOPS/W at 50 MHz/0.56 V, and the peak performance is 230.4 GOPS at 200 MHz/1.1 V, without considering sparsity.

The testing platform is shown in Fig. 12. An field-programmable gate array (FPGA) board (Xilinx Virtex UltraScale+) is connecting between the host PC and the chip-mounting PCB board. Testing data including test program and model data are sent from the PC to FPGA memories. The accelerator will load data from or store data in FPGA memories. Once the computation is completed, the PC reads back data from the FPGA to validate the correctness and also measures the computation latency. The accelerator has an independent power supply from a Keysight B2902A dc power analyzer, and the power consumption of the core can be measured from it.

B. Real Network Efficiency

To evaluate the efficiency of the proposed accelerator, we use the Spikingformer network [16] and a Spiking ResNet [17] as benchmarks as shown in Table I. The weights are quantized to 8 bits. The suffix “8–768” in Spikingformer represents the network architecture, which denotes that the network uses eight transformer blocks with a hidden state dimension of 768, and the “9–32” in spiking ResNet denotes nine residual layers are used with a channel number of 32. Three datasets, including two static image datasets ImageNet and CIFAR-10, and one dynamic vision dataset DVS-128 for gesture recognition, are used in this work. The input image

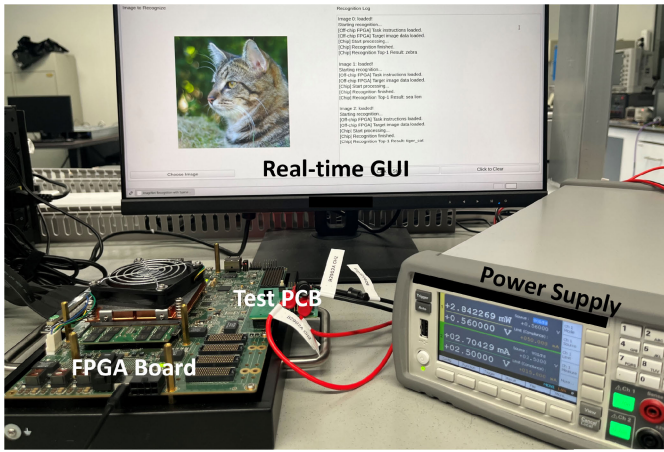


Fig. 12. Photograph of the testing platform, including a host PC, an FPGA board, and a dc power supply.

TABLE I

END-TO-END MEASUREMENT RESULTS FOR DIFFERENT MODELS

Network	Spikeformer-8-768	Spikeformer-2-384	Spiking ResNet-9
Dataset	ImageNet	CIFAR-10	DVS128
¹ Sparse Level	82.9%	87.2%	91.5%
Accuracy	77.6%	95.2%	96.1%
² Latency	304.6 ms	10.18 ms	2.18 ms
² FPS	3.28	98.24	459.7
² Energy Efficiency	42.8 TOPS/W	49.7 TOPS/W	65.2 TOPS/W

¹The average spike sparsity in each layer, weights are dense; ²Measured at 50MHz, 0.56V. IO time included.

data in the static dataset will be encoded into spike format with one convolution layer a spiking neuron layer. For the dynamic dataset, the raw sequence data are already in spike format. We adopt a preprocessing procedure consistent with [17] and [8], segmenting the raw spikes into $T = 8$ timesteps and accumulating the spikes to get the spike number. An additional conv layer will encode the spike number into spike format again. Note that the additional convolution layer is non-spiking, and it is thus implemented on FPGA, but its latency is also included in measurement results. We achieved an average energy efficiency from 42.8 to 65.2 TOPS/W in the software benchmark. The variation of energy efficiency in different networks is largely dependent on the average sparsity level of the network, which is consistent with the results shown in Fig. 9(b). The breakdown of the energy efficiency of each layer in Spikingformer-8-768 on the ImageNet dataset is shown in Fig. 13(a). The variation is also dependent on the sparsity level of each layer as shown in Fig. 13(b). The energy efficiency of SSA is typically higher than other operators not only because of the high spike sparsity in the Q and K matrix (up to 97%) but also because the implementation of SAS greatly reduces the total amount of operations in SSA.

C. Comparison With State-of-the-Art Accelerators

Table II shows the comparison between the proposed accelerator and other state-of-the-art processors, including both SNN processors and sparse ANN accelerators. To make a fair comparison, it is essential to first define the energy efficiency in SNN operations. Following the conventions in [1] and [6],

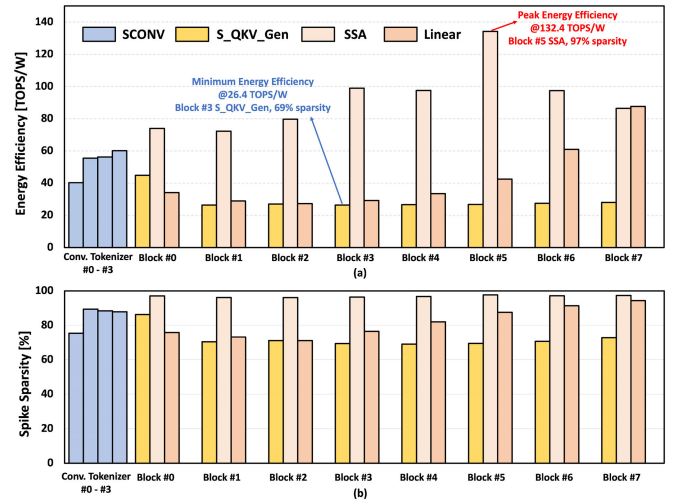


Fig. 13. (a) Per-layer energy efficiency of Spikingformer-8-768 on ImageNet. (b) Per-layer input spike sparsity of Spikingformer-8-768 on ImageNet.

we used both TOPS/W and pJ/SOP for energy efficiency evaluations. For TOPS/W, we define 2OPs to represent eight spike ACs of 8-b weights, because 2OPs represent one 8-b MAC in conventional ANNs. This definition is also consistent with existing SNN processor that also uses TOPS/W [1], [2], [6]. The definition also makes sense from an energy consumption perspective, because the power consumed by one 8-b MAC is about 7.7 times of an 8-b AC under 45-nm technology [18], [19]. For pJ/SOP, we follow the definition in [6], which is obtained by the total consumed power consumption for a sample divided by the total amount of spike operation counts in the network.

While our work adopts a spike-only homogeneous design, some of the SNN accelerators propose to use a heterogeneous architecture, allowing both SNN or ANN computation on the chip to obtain a higher energy efficiency [2], [6]. Such an approach is only applicable in networks that apply ANN2SNN conversion, where operations in the network can be either represented by the conventional MAC or by the converted spiking ACs. Conversion-based SNNs typically have long timesteps and that is the reason why the energy efficiency of SNN could even fall below the efficiency of ANN when sparsity is low [2], [6], [18]. We adopt the direct-trained SNN where all information is in spike format and cannot revert back to an ANN representation. Direct-trained networks have fewer timesteps than conversion-based SNNs without loss of accuracy. For example, only four timesteps are required to obtain an accuracy of 80.0% in ImageNet for SOTA direct trained SNNs [11], while conversion-based network can only achieve 78.51% but with 512 timesteps [20]. In addition, with the optimizations applied by this work, the processor can achieve a higher energy efficiency than ANN energy efficiency baseline in [6] at all spike sparsity as shown in Fig. 9. Therefore, only the AC array required by SNN is implemented, saving the cost of implementing MAC arrays. As a result, our work achieves an energy efficiency of 42.8 TOPS/W on the ImageNet dataset, which is $1.75\times$ higher energy efficiency

TABLE II
COMPARISON OF SNN PROCESSOR AND SPARSE ANN PROCESSORS

	THIS WORK	ISSCC'24 [2]	ISSCC'23 [6]	JSSC'23 [1]	ISSCC'23 [5]	ISSCC'23 [21]	ISSCC'23 [22]
Technology [nm]	40	28	28	28	28	28	28
Area [mm ²]	2.24	20.25	20.25	6.3	1.25	2.69	7.29
Frequency [MHz]	50-200	50-200	50-200	2-200	40-210	40 - 450	20 - 400
Activation Precision ⁽¹⁾	INT 1-8	INT8	INT 1-16	INT 1-8	INT 1	INT 8	AQ 1-8
Weight Precision	INT 8	INT8	INT4/8	INT2/4/8	INT8/INT10	INT 8	AB 1-8
Sparsity Support	Input	Input	Input	Input + Weight	Input + Weight	Input	Input + Weight
Support Network	Spiking Transformers /Spiking ResNet	Converted Spiking Transformers	Spiking CNN/CNN	Spiking CNN	Spiking MLP	CNN	CNN/Transformers
Peak Performance ⁽²⁾ [TOPS]	0.23 - 1.50 @INT8/8	3.41 @INT8	3.28 @INT8/8	0.82 - 5.32 @INT8/8	0.22 @INT1/8	0.23 - 0.76 @INT8/8	0.52-5.26 @AQ8/AB8
Energy efficiency	20.28 ⁽³⁾ - 132.42 ⁽⁵⁾ TOPS/W Avg: 42.8 ⁽⁴⁾ TOPS/W 0.078 ⁽⁴⁾ pJ/SOP @INT8/8	22.9 - 47.8 TOPS/W @INT8/8	Avg: 24.5 TOPS/W 1.1 ⁽⁴⁾ pJ/SOP @INT8/8	6.27-117.77 ⁽⁵⁾ TOPS/W 0.35 pJ/SOP @INT8/8	1.49pJ/SOP @INT1/8	1.55 - 11.99 ⁽⁷⁾ TOPS/W @INT8/8	8.94 - 127.8 ⁽⁶⁾ TOPS/W Avg: 30.25 TOPS/W @AQ8/AB8
Area Efficiency [GOPS/mm ²]	102.9 - 669.8	168.4	161.7	130.0 - 844.4	175.4	85.5 - 282.5	71.3 - 721.5
Dataset & Model & Accuracy	ImageNet: Spikingformer-8- 768 (77.6%) CIFAR-10: Spikingformer-2-384 (95.2%) DVS128: ResNet-9 (96.1%)	GPT-2: Wikitext mT5: IWSLT T5: CNN/DM FSMT: IWSLT	ImageNet: ResNet-50 (77.1%) CIFAR-10: VGG16 (94.1%) CIFAR-100: ResNet-18 (75.9%)	ImageNet: ResNet-18 (66.8%) CIFAR-10: ResNet-12 (89.2%) COCO2014: Tiny-YOLO v3 (29.5 mAP)	N-MNIST: 96.0% DVS128: 92.0% KWS: 92.6% SeNic: 95.7%	ImageNet: ResNet-34 (71.8%) KITTI: SECOND (78.49mAP)	ImageNet: Eff-N-L0 (75.1%) ViT-T (75.5%) M.Mxr-B (76.6%)
Task Energy	ImageNet: 0.85mJ ⁽⁸⁾ CIFAR-10: 0.029mJ ⁽⁸⁾ DVS128: 6.2μJ ⁽⁸⁾	GPT-2: 18.1μJ/token mT5: 10.4μJ/token T5: 7.2μJ/token FSMT: 2.6μJ/token	N/A	ImageNet: 1.95mJ CIFAR10: 0.27mJ COCO2014: 2.95mJ	NMNIST: 343nJ DVS128: 3.9μJ ⁽⁹⁾ KWS: 6.1μJ SeNic: 582nJ	ImageNet: 0.57mJ KITTI: 1.9mJ	N/A

¹Bit counts to store a pixel of an activation. ²TOPs = 1 MAC = (number of timesteps) accumulation; In case of INT 8/8, the energy consumption of eight 8b accumulations is equivalent to one 8b-MAC.

³Measured at 0.56V/50MHz, from 0% to 97% spike sparsity ⁴pJ/SOP = (Energy/frame) / (total number of spikes) ⁵97% spike sparsity & 90% weight sparsity ⁶90% input sparsity & 90% weight sparsity

⁷40% input sparsity ⁸Measured at 0.56V/50MHz, assumed enough EMA bandwidth, the first non-spiking conv layer is excluded. ⁹Downscaled to 2x17x17, 10 classed.

than SOTA SNN processors [6]. In addition, the input sparsity exploitation is also adopted in other SNN processors [1], [2], [6]. For example, in [6], the shift-AC PE will be **clock-gated to save power** if consecutive input spikes in a time window are 0. While in this work, **only non-zero elements are routed** to the computation engine continuously to ensure a high hardware utilization. Compared with the gating method, the proposed non-zero data fetching technique has a higher power consumption but greatly reduces computation latency to offset the increased power. As a result, 56.4% lower energy is required to classify each image in ImageNet than [1] in our work and still achieving the highest accuracy of 77.7% among all relevant works **thanks to the use of the SOTA spiking transformer algorithms**.

We also compare our work with some SOTA sparse ANN processors. ANN cannot achieve a level of input sparsity as high as SNN, and a more costly MAC array is required. Therefore, **the sparse ANN processors that utilize only the input sparsity can only achieve an energy efficiency of 11.99 TOPS/W on ImageNet, because at most only 40% input sparsity can be achieved [21]**. We can achieve a 3.57× higher energy efficiency. With the use of weight pruning, ANN can achieve up to 90% of input and weight sparsity. However, the accuracy of the network would be affected by aggressive pruning, and the logic required to exploit both sparsity becomes complex and would be difficult to scale up [12], [13]. Therefore, although [22] exploits both input and weight sparsity on ImageNet and achieves an energy efficiency of 30.57 TOPS/W, our work still achieves 1.4× higher energy efficiency. In addition, thanks to the use of spiking transformers, the accuracy obtained on ImageNet is even higher than that on ANN processors. It is strong proof

that the accuracy of SNN is already comparable with ANN of the same scale, but with a competitive feature in energy-efficient processing.

VIII. CONCLUSION

This article proposes the first sparse accelerator that provides support and acceleration for multiple direct-trained DSNs like spiking transformers and spiking ResNet. A 3-D computation array is implemented to support parallel computation among timesteps that greatly alleviates redundant memory access of weights and partial sums. In addition, a high-throughput non-zero data fetcher along with a two-staged fan-in prioritized dataflow is proposed to efficiently exploit the unstructured input spike sparsity in SNN. Finally, a unified scheduler is proposed to provide support for three unique operators including SCONV, S_QKV_Gen, and SSA, which further reduce processing latency with optimized dataflows. The chip is fabricated in 40-nm CMOS technology and evaluated with static and dynamic image datasets. It achieves an energy efficiency of 0.078 pJ/SOP and 42.8 TOPS/W on ImageNet, which is 1.75× higher than SOTA SNN processors and 1.4× higher than sparse ANN processors.

REFERENCES

- [1] S. Kim, S. Kim, S. Um, S. Kim, J. Lee, and H.-J. Yoo, "SNPU: An energy-efficient spike domain deep-neural-network processor with two-step spike encoding and shift-and-accumulation unit," *IEEE J. Solid-State Circuits*, vol. 58, no. 10, pp. 2812–2825, Oct. 2023.
- [2] S. Kim, S. Kim, W. Jo, S. Kim, S. Hong, and H.-J. Yoo, "C-transformer: A 2.6–18.1 μJ/token homogeneous DNN-transformer/spiking-transformer processor with big-little network and implicit weight generation for large language models," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 368–370.

- [3] G. K. Chen, R. Kumar, H. E. Sumbul, P. C. Knag, and R. K. Krishnamurthy, "A 4096-neuron 1M-synapse 3.8-pJ/SOP spiking neural network with on-chip STDP learning and sparse weights in 10-nm FinFET CMOS," *IEEE J. Solid-State Circuits*, vol. 54, no. 4, pp. 992–1002, Apr. 2019.
- [4] C. Frenkel and G. Indiveri, "ReckOn: A 28 nm sub-mm² task-agnostic spiking recurrent neural network processor enabling on-chip learning over second-long timescales," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.
- [5] J. Zhang et al., "ANP-I: A 28 nm 1.5 pJ/SOP asynchronous spiking neural network processor enabling sub-0.1 μ J/sample on-chip learning for edge-AI applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, 2023, pp. 21–23.
- [6] S. Kim, S. Kim, S. Hong, S. Kim, D. Han, and H.-J. Yoo, "C-DNN: A 24.5–85.8TOPS/W complementary-deep-neural-network processor with heterogeneous CNN/SNN core architecture and forward-gradient-based sparsity generation," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 334–336.
- [7] M. Yao et al., "Spike-driven transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–16.
- [8] Z. Zhou et al., "Spikformer: When spiking neural network meets transformer," in *Proc. 11th Int. Conf. Learn. Represent.*, 2022, pp. 1–19.
- [9] Q. Su et al., "Deep directly-trained spiking neural networks for object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 6532–6542.
- [10] Z. Wang, Y. Zhang, S. Lian, X. Cui, R. Yan, and H. Tang, "Toward high-accuracy and low-latency spiking neural networks with two-stage optimization," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Dec. 15, 2023, doi: [10.1109/TNNLS.2023.3337176](https://doi.org/10.1109/TNNLS.2023.3337176).
- [11] M. Yao et al., "Spike-driven transformer v2: Meta spiking neural network architecture inspiring the design of next-generation neuromorphic chips," 2024, *arXiv:2404.03663*.
- [12] Z. Yuan et al., "STICKER: An energy-efficient multi-sparsity compatible accelerator for convolutional neural networks in 65-nm CMOS," *IEEE J. Solid-State Circuits*, vol. 55, no. 2, pp. 465–477, Feb. 2020.
- [13] J.-F. Zhang, C.-E. Lee, C. Liu, Y. S. Shao, S. W. Keckler, and Z. Zhang, "SNAP: An efficient sparse neural acceleration processor for unstructured sparse deep neural network inference," *IEEE J. Solid-State Circuits*, vol. 56, no. 2, pp. 636–647, Feb. 2021.
- [14] A. Parashar et al., "SCNN: An accelerator for compressed-sparse convolutional neural networks," *ACM SIGARCH Comput. Archit. News*, vol. 45, no. 2, pp. 27–40, Jun. 2017.
- [15] A. Dosovitskiy et al., "An image is worth 16 $\times$ 16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [16] C. Zhou et al., "Spikingformer: Spike-driven residual learning for transformer-based spiking neural network," 2023, *arXiv:2304.11954*.
- [17] W. Fang, Z. Yu, Y. Chen, T. Huang, T. Masquelier, and Y. Tian, "Deep residual learning in spiking neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 21056–21069.
- [18] M. Dampfhofer, T. Mesquida, A. Valentian, and L. Anghel, "Are SNNs really more energy-efficient than ANNs? An in-depth hardware-aware study," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 3, pp. 731–741, Jun. 2023.
- [19] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2014, pp. 10–14.
- [20] Z. Wang, Y. Fang, J. Cao, Q. Zhang, Z. Wang, and R. Xu, "Masked spiking transformer," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2023, pp. 1761–1771.
- [21] W. Sun et al., "A 28 nm 2D/3D unified sparse convolution accelerator with block-wise neighbor searcher for large-scaled voxel-based point cloud network," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 328–330.
- [22] S. Moon, H.-G. Mun, H. Son, and J.-Y. Sim, "A 127.8 TOPS/W arbitrarily quantized 1-to-8b scalable-precision accelerator for general-purpose deep learning with reduction of storage, logic and latency waste," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 21–23.



Chaoming Fang (Member, IEEE) received the B.S. degree in instrumentation science and technology from Beihang University, Beijing, China, in 2021. He is currently pursuing the Ph.D. degree with the Center of Excellence in Biomedical Research on Advanced Integrated-on-Chips Neurotechnologies (CenBRAIN Neurotech), School of Engineering, Westlake University, Hangzhou, China.

He joined CenBRAIN Neurotech, School of Engineering, Westlake University, in 2021. His current research interests include hardware-software co-design for neuromorphic computing, VLSI design for deep learning accelerator, and domain-specific processor design for edge applications.



Ziyang Shen (Member, IEEE) received the B.Eng. degree in microelectronics from Fudan University, Shanghai, China, in 2022. He is currently pursuing the Ph.D. degree with the Center of Excellence in Biomedical Research on Advances-on-Chips Neurotechnologies (CenBRAIN Neurotech), Westlake University, Hangzhou, China.

His research interests include spike-based neuromorphic vision algorithm and domain-specific neuromorphic accelerator design for edge applications.



Zongsheng Wang (Member, IEEE) received the bachelor's degree from the College of Electrical Engineering, Zhejiang University, Hangzhou, China, in 2020, where he is currently pursuing the master's degree with the College of Information Science and Electronic Engineering.

He is interested in in-sensor computing, low-power digital circuit design, and deep learning accelerator.



Chuanqing Wang (Member, IEEE) received the B.S. degree from Harbin Engineering University, Harbin, China, in 2019, and the joint Ph.D. degree from the electronic science and technology, Fudan University, Shanghai, China, and Westlake University, Hangzhou, China, in 2024.

He is currently an Associate Researcher with the Institute of Energy, Hefei Comprehensive National Science Center, Hefei, China. His research interests include retinal prosthesis, brain-inspired computing, spiking neural networks, and AI for science with a special focus on battery and catalyst.



Shiqi Zhao (Member, IEEE) received the joint Ph.D. degree in electronic science and technology from Zhejiang University, Hangzhou, China, and Westlake University, Hangzhou, in 2023.

He is currently an Assistant Professor with the School of Computer and Communication Engineering, Northeastern University, Shenyang, China. His current research interests include brain-machine interface, energy-efficient deep learning algorithm, customized computer architecture design, and hardware implementation for biomedical deep learning processors.



Fengshi Tian (Member, IEEE) received the B.Eng. degree in microelectronics science and engineering from Fudan University, Shanghai, China, in 2021. He is currently pursuing the Ph.D. degree with the ECE Department, AI Chip Center for Emerging Smart System (ACCESS), The Hong Kong University of Science and Technology (HKUST), Hong Kong, SAR.

He was with the Center of Excellence in Biomedical Research on Advances-on-Chips Neurotechnologies (CenBRAIN Neurotech), Westlake University, Hangzhou, China, as a Visiting Student, from 2020 to 2021. He was also a Visiting Student with the University of Oxford, Oxford, U.K., in 2019 and Waseda University, Shinjuku, Japan, in 2018. He also had multiple experiences in IC industry and investment business. His current research interests lie in neuromorphic computing for edge applications, domain-specific architecture design, and computing-in-memory systems.



Jie Yang (Member, IEEE) received the B.S. degree in electronic science and technology from Tianjin University, Tianjin, China, in 2010, and the Ph.D. degree in microelectronics from the Institute of Semiconductors, Chinese Academy of Sciences, Beijing, China, in 2015.

From 2015 to 2019, he was a Post-Doctoral Fellow with the I2Sense Laboratory, Department of Electrical and Computer Engineering, University of Calgary, Calgary, AB, Canada. In 2019, he joined the Center of Excellence in Biomedical Research on Advances-on-Chips Neurotechnologies (CenBRAIN Neurotech), School of Engineering, Westlake University, Hangzhou, China, where he is currently a Research Professor. His research interests include integrated circuits for brain-computer interfaces, energy-efficient neuromorphic computing, and brain-language decoding.



Mohamad Sawan (Life Fellow, IEEE) received the Ph.D. degree from the University of Sherbrooke, Sherbrooke, QC, Canada.

He is currently a Chair Professor with Westlake University, Hangzhou, China, and an Emeritus Professor with Polytechnique Montreal, Montréal, QC, Canada. He is also the Founder and the Director of the Center of Excellence in Biomedical Research on Advances-on-Chips Neurotechnologies (CenBRAIN Neurotech), Westlake University, and the Polystim Neurotech Laboratory, Polytechnique Montreal, Montréal. He has authored or co-authored more than 1000 peer-reviewed articles, one handbook, three books, 13 book chapters, 12 patents, and 20 other patents are pending.

Dr. Sawan is a fellow of the Royal Society of Sciences of Canada (FRSC), Canadian Academy of Engineering (FCAE), the Engineering Institutes of Canada (FEIC), and the Asia-Pacific Artificial Intelligence Association (FAAIA); and an "Officer" of the National Order of Quebec. He was awarded the Canada Research Chair in Smart Medical Devices from 2001 to 2015. He received several awards, among them the Barbara Turnbull Award from the Canadian Institutes of Health Research (CIHR), the J. A. Bombardier and Jacques-Rousseau Awards from the Canadian ACFAS, the Queen Elizabeth II Golden Jubilee Medal, the Medal of Merit from the President of Lebanon, and the Shanghai International Collaboration Award. He served as a Distinguished Lecturer for IEEE Circuit and System Society (CASS) and IEEE Solid States Circuits Society (SSCS) for three consecutive years each. He was leading the Microsystems Strategic Alliance of Quebec (ReSMiQ), Canada, from 1999 to 2018. He was a Co-Founder, an Associate Editor, and an Editor-in-Chief of IEEE TRANSACTIONS ON BIOMEDICAL CIRCUITS AND SYSTEMS from 2016 to 2019.